



Universidade Federal
de São João del-Rei

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL REI

Departamento de Ciência da Computação

Trabalho Prático 1 de Algoritmos BioInspirados

Gabriel Vaz Bernardini Ferreira - 2023008344

Felipe Girardi Siqueira - 2023008120

Lucas Daniel Lana Maciel - 2023008362

São João Del Rei

Abril de 2026

Sumário

1	Introdução	2
2	Metodologia	2
2.1	Estrutura do Algoritmo genético	3
3	Configurações Experimentais	4
4	Resultados e Discussão	5
4.1	Problema LAU15	5
4.2	Problema SGB128	7
5	Conclusão	9

1 Introdução

O Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP) é um dos problemas clássicos de otimização combinatória, amplamente estudado na literatura devido à sua complexidade e relevância prática. O objetivo do TSP consiste em determinar a menor rota possível que permita a um viajante visitar um conjunto de cidades exatamente uma vez e retornar à cidade de origem.

Esse problema pertence à classe dos problemas NP-difíceis, o que implica que, para instâncias de maior porte, métodos exatos tornam-se computacionalmente inviáveis. Dessa forma, abordagens heurísticas e metaheurísticas têm sido amplamente empregadas para obter soluções de boa qualidade em tempo razoável.

Entre essas abordagens, destacam-se os Algoritmos Genéticos (AG's), inspirados nos princípios da evolução natural como seleção, cruzamento e mutação. Esses algoritmos são particularmente adequados para problemas como o TSP, pois trabalham naturalmente com representações baseadas em permutações e permitem explorar eficientemente grandes espaços de busca.

No presente trabalho, desenvolve-se e avalia-se um Algoritmo Genético aplicado ao TSP, com foco na análise do impacto de diferentes operadores de cruzamento e parâmetros como taxa de mutação, tamanho da população e número de gerações. Além disso, são realizados experimentos fatoriais para comparar o desempenho e a estabilidade das configurações testadas.

2 Metodologia

Sendo assim, com o conhecimento sobre o problema do Caixeiro Viajante, adotou-se a seguinte modelagem para o problema:

1. Representação do indivíduo: Cada indivíduo é representado por um vetor, que indica a ordem de visita do indivíduo em cada cidade. Ex: $[0, 3, 2, 1]$. Neste exemplo, o indivíduo visitou respectivamente as cidades zero, três, dois e um.
2. Função objetivo: Como a missão é minimizar a distância total e, percorrer todas as cidades exatamente uma vez até a cidade de origem ser a mesma que a de destino, há o cálculo total da distância percorrida por cada indivíduo, assim é considerado o indivíduo com a menor distância total, o valor ótimo do problema.
3. Função de fitness: Como a função, por padrão, no algoritmo genético é de maximização, para transformá-la em minimização, passou-se o valor negativo da distância total percorrida.

2.1 Estrutura do Algoritmo genético

A implementação foi estruturada de forma modular, permitindo fácil alteração de parâmetros e operadores, facilitando a realização de experimentos fatoriais. Com isso, a inicialização da população foi feita de forma aleatória, e cada indivíduo é uma permutação válida das cidades, indicando a ordem de visita.

Ao longo das gerações, novos indivíduos são produzidos a partir da aplicação de operadores genéticos, com isso em mente o modo de seleção dos pais é por meio do torneio, onde um subconjunto da população é escolhido aleatoriamente e o melhor indivíduo desse grupo é selecionado para a reprodução, favorecendo indivíduos mais aptos e mais diversificados.

Depois da escolha dos pais, o cruzamento é aplicado de acordo com uma porcentagem e implementado aceitando dois tipos diferentes: Order Crossover (OX) e Partially Mapped Crossover (PMX).

O operador OX preserva a ordem relativa dos elementos, copiando um segmento de um dos pais e preenchendo o restante com base na sequência de outro pai, evitando repetições. Já o operador PMX estabelece um mapeamento entre os segmentos dos pais, garantindo a consistência da permutação por meio de substituições baseadas nesse mapeamento.

Após o cruzamento, a mutação é aplicada, essa técnica consiste na inversão de um segmento da rota, assim selecionando dois pontos aleatórios e invertendo a sequência entre eles, diversificando os indivíduos e mitigando a convergência precoce em ótimos locais, junto com essas técnicas, foi adotada a estratégia de elitismo, que preserva uma porção dos melhores pais da geração anterior para que o algoritmo não volte para soluções piores do que as encontradas anteriormente.

Assim, todas essas funções iteram de acordo com a quantidade de gerações, durante a execução, no terminal, há uma prévia de resultados encontrados por cada iteração dentro do loop principal.

Com essa execução base, foi implementada uma busca em grade (grid search) para a análise de comportamento da convergência do algoritmo em diferentes configurações de variáveis, nesse processo, há diversas execuções com as diferentes configurações e a troca de valores nas seguintes variáveis: tamanho da população, número de gerações, taxa de mutação, taxa de cruzamento e operador de cruzamento.

Portanto, ao final da execução, os resultados como a configuração, indivíduo (rota) e distância total calculada são armazenados individualmente e foram calculadas outras métricas estatísticas para cada configuração como: melhor resultado, pior resultado, média, mediana e desvio padrão, permitindo avaliar tanto o desempenho quanto a estabilidade do algoritmo.

Por fim, os dados obtidos são utilizados para a geração de gráficos, incluindo gráficos boxplot para análise de dispersão da dispersão dos resultados, gráficos de barras com

média e desvio padrão para comparação de configurações e curvas de convergência.

3 Configurações Experimentais

Os experimentos foram conduzidos utilizando o Algoritmo Genético apresentado no capítulo anterior. Como o objetivo do estudo é analisar o impacto de diferentes parâmetros no desempenho do algoritmo, foi realizado um teste fatorial completo, considerando múltiplos valores para cada parâmetro.

A seguir, são descritos os parâmetros utilizados:

- **POP_SIZE**: tamanho da população, ou seja, a quantidade de indivíduos (rotas) em cada geração.
- **GENERATIONS**: número de gerações executadas pelo algoritmo, representando o critério de parada.
- **MUTATION_RATE**: taxa de mutação aplicada aos indivíduos, responsável por introduzir diversidade no processo evolutivo.
- **CROSSOVER_OPERATOR**: operador de cruzamento utilizado na recombinação dos indivíduos. Foram avaliados os operadores OX (Order Crossover) e PMX (Partially Mapped Crossover).
- **CROSSOVER_RATE**: probabilidade de aplicação do operador de cruzamento entre dois indivíduos selecionados.
- **iterations**: número de execuções independentes realizadas para cada configuração de parâmetros, com diferentes sementes aleatórias, visando garantir a robustez estatística dos resultados.

O conjunto de valores adotado para cada parâmetro está apresentado na Tabela 1.

POP_SIZE	MUTATION-RATE	CROSSOVER-OPERATOR	CROSSOVER-RATE
50	0.01	OX	0.8
100	0.05	PMX	0.9

Tabela 1: Tabela com a lista de parâmetros estudados

A combinação de todos os valores dos parâmetros resulta em um total de 32 configurações distintas (2 valores para cada um dos 5 parâmetros), sendo cada uma executada 10 vezes de forma independente, cada uma com uma seed diferente. Dessa forma, foram realizadas 320 execuções do algoritmo, permitindo uma análise estatística consistente dos resultados que será discutida no próximo capítulo.

4 Resultados e Discussão

Os experimentos foram realizados considerando diferentes combinações de parâmetros do algoritmo genético, avaliando o desempenho por meio de métricas como melhor resultado, média e desvio padrão ao longo de 10 execuções independentes.

Os resultados obtidos a partir dessas execuções foram organizados em dois conjuntos principais: resultados individuais de cada execução e estatísticas agregadas por configuração de parâmetros.

Antes de prosseguir com a análise é válido destacar que o número de gerações foi fixo para cada problema, sendo 100 gerações para o *LAU15* e 300 gerações para o *SGB128*. Além disso, para facilitar ao leitor, todas as configurações de número par representam o operador de *crossover* *OX*, já os ímpares do *PMX*. Portanto, segue a tabela com todas as combinações enumeradas:

ID-CONFIG	POP_SIZE	MUTATION-RATE	CROSSOVER-OPERATOR	CROSSOVER-RATE
1	50	0.01	OX	0.8
2	50	0.01	PMX	0.8
3	50	0.01	OX	0.9
4	50	0.01	PMX	0.9
5	50	0.05	OX	0.8
6	50	0.05	PMX	0.8
7	50	0.05	OX	0.9
8	50	0.05	PMX	0.9
9	100	0.01	OX	0.8
10	100	0.01	PMX	0.8
11	100	0.01	OX	0.9
12	100	0.01	PMX	0.9
13	100	0.05	OX	0.8
14	100	0.05	PMX	0.8
15	100	0.05	OX	0.9
16	100	0.05	PMX	0.9

4.1 Problema LAU15

Analisando os melhores resultados de cada uma das 16 combinações podemos ver que 10 chegaram em um resultado ótimo (291) com configurações bem mescladas tanto de operadores de cruzamento quanto da taxa de *crossover* e mutações. Das configurações que não alcançaram o ótimo, 3 delas ficaram abaixo da distância 300 e as outras 3 abaixo do 340. Segue, na próxima página, um gráfico com os *boxplots* dos resultados de cada configuração.

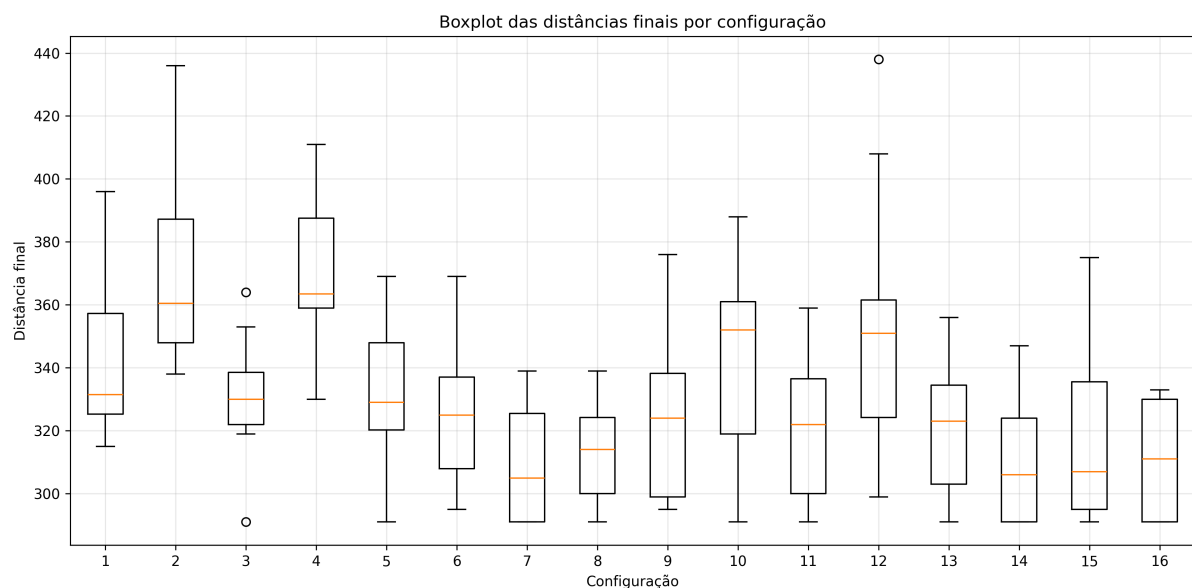


Figura 1: Boxplots dos resultados LAU 15

A partir dos *boxplots* gerados, foi possível observar a presença de *outliers* em diversas configurações, principalmente nas que utilizam o operador *PMX* e maiores taxas de mutação.

Em contraste, a maioria das configurações com o operador *OX* apresentaram distribuições mais compactas, com menor dispersão entre os valores mínimo e máximo e, consequentemente, menor ocorrência de *outliers*. Isso indica que o *OX* tende a produzir resultados mais consistentes entre diferentes execuções. Porém é importante destacar que esses *outliers* de ambas configurações podem ser interpretados como falhas ocasionais do algoritmo em convergir para regiões promissoras do espaço de busca, possivelmente devido a combinações desfavoráveis entre cruzamento e mutação.

Seguindo a análise visual dos *boxplots* juntamente com os dados brutos, é importante destacar que algumas configurações, embora tenham atingido o valor ótimo em pelo menos uma execução, apresentaram alta variabilidade nos resultados. Isso indica que tais configurações possuem boa capacidade de exploração, porém menor estabilidade, dependendo fortemente da inicialização aleatória. Por outro lado, configurações com caixas mais compactas e menor dispersão demonstram maior robustez, mesmo quando não atingem o ótimo em todas as execuções.

Nesse contexto, a utilização da mediana (linha amarela nos *boxplots* do gráfico 1) como métrica de avaliação se mostra mais adequada do que a média, uma vez que a presença de *outliers* pode distorcer significativamente o valor médio. A mediana, por sua vez, representa melhor o comportamento típico das execuções.

4.2 Problema SGB128

A partir da análise dos dados resumidos, observa-se que diferentes configurações de parâmetros impactam significativamente tanto a qualidade das soluções quanto a estabilidade do algoritmo, contrastando as soluções obtidas no problema anterior. Comparando inicialmente os operadores de cruzamento, nota-se que o operador *OX* apresentou desempenho significativamente superior ao *PMX*. Enquanto configurações com *OX* atingiram distâncias médias na faixa de aproximadamente 47.000 a 50.000, as configurações com *PMX* apresentaram médias muito mais elevadas, superiores a 90.000, indicando soluções consideravelmente piores.

Além disso, o desvio padrão das execuções com *OX* tende a ser menor, o que indica maior estabilidade nas soluções encontradas. Por outro lado, o operador *PMX* apresentou maior variabilidade, com desvios superiores a 5000 em algumas configurações, sugerindo menor consistência entre execuções.

Em relação à taxa de cruzamento, observa-se que valores mais elevados (0.9) tendem a produzir melhores resultados quando combinados com o operador *OX*, indicando que uma maior exploração por recombinação contribui positivamente para a qualidade das soluções.

No que diz respeito à taxa de mutação, seu impacto mostrou-se mais sutil. Comparando configurações com taxa de mutação de 0.01 e 0.05, observa-se que valores maiores podem contribuir para uma leve melhora na estabilidade (redução do desvio padrão), possivelmente devido ao aumento da diversidade populacional e à mitigação de convergência prematura.

A análise dos resultados também evidencia a importância da execução de múltiplas rodadas para cada configuração. Mesmo com os mesmos parâmetros, diferentes sementes aleatórias geraram variações consideráveis nos resultados individuais, o que reforça a necessidade do uso de métricas estatísticas como média, mediana e desvio padrão para uma avaliação mais robusta.

O gráfico 2 mostra a diferença dos resultados analisados anteriormente e a extensão da variedade de piores casos como apresentado no índice 2 e 4, ambos *PMX*, que são valores fora do padrão se comparados com as outras configurações.

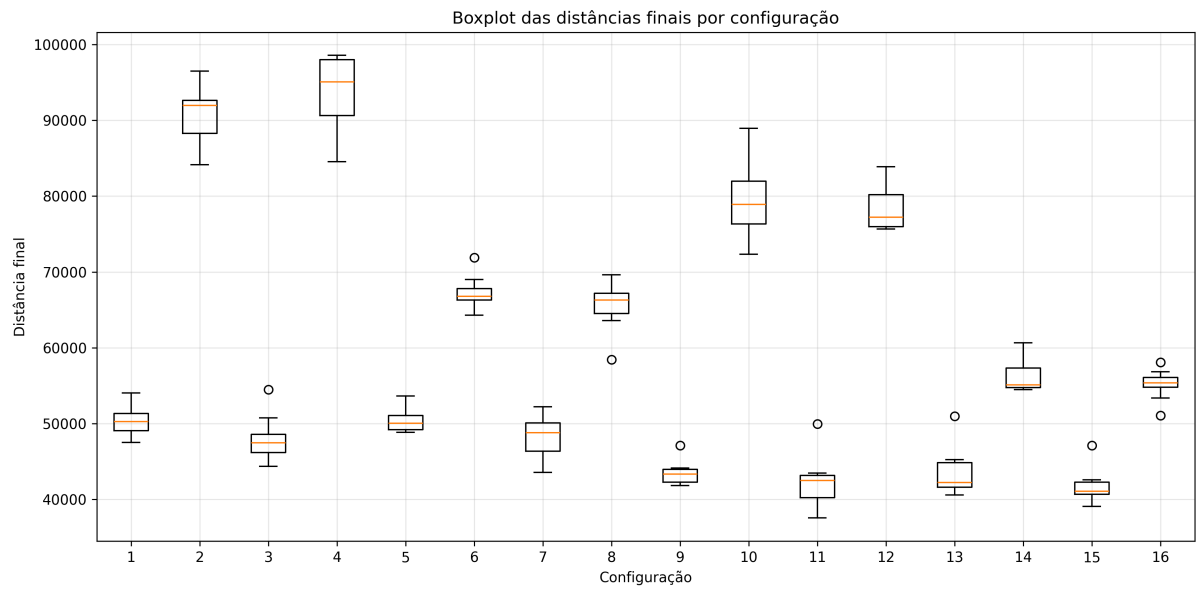


Figura 2: Gráfico boxplot - Problema SGB128

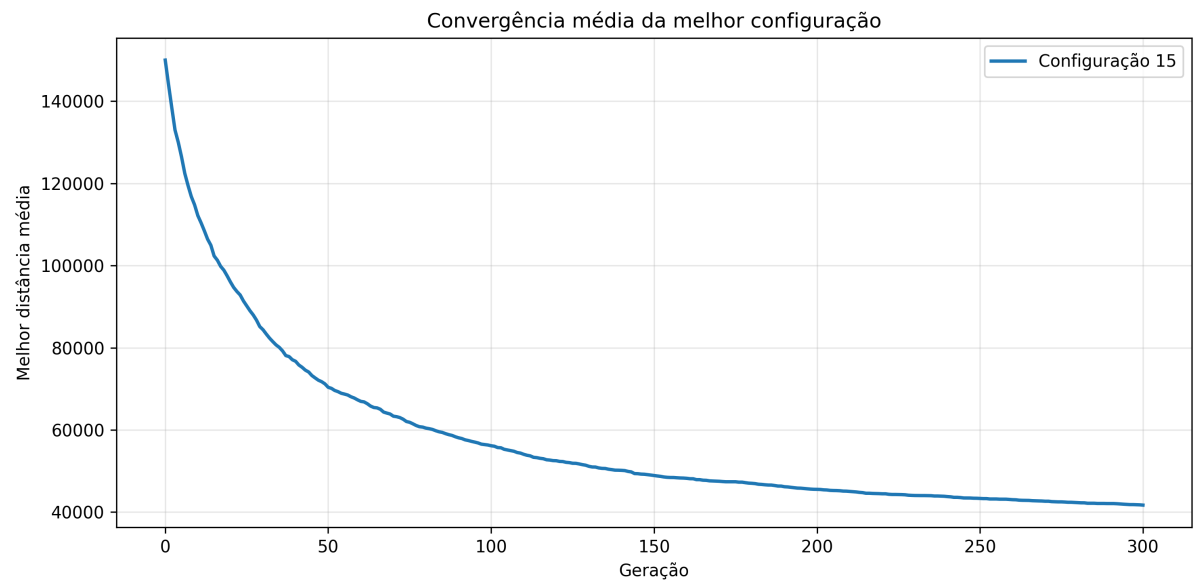


Figura 3: Gráfico de convergência da melhor configuração - SGB128

Por fim, a análise da curva de convergência da melhor configuração mostra que o algoritmo apresenta uma redução significativa da distância nas primeiras gerações, seguida de uma estabilização progressiva, indicando convergência para uma solução próxima do ótimo. Esse comportamento é consistente com o esperado para algoritmos genéticos aplicados a problemas de otimização combinatória.

5 Conclusão

Este trabalho teve como objetivo analisar o desempenho de um Algoritmo Genético aplicado ao Problema do Caixeiro Viajante, considerando diferentes configurações de parâmetros e operadores genéticos. A partir dos experimentos realizados, foi possível observar como essas variações impactam tanto a qualidade das soluções quanto a estabilidade do algoritmo.

A comparação entre os problemas LAU15 e SGB128 evidenciou claramente o efeito da complexidade do problema sobre o comportamento do algoritmo. No caso do LAU15, por se tratar de uma instância pequena, diversas configurações foram capazes de encontrar a solução ótima, indicando que o problema é relativamente fácil de resolver e pouco sensível à escolha dos parâmetros. Por outro lado, no problema SGB128, significativamente mais complexo, as diferenças entre as configurações tornaram-se muito mais evidentes, destacando a importância da escolha adequada dos operadores e parâmetros.

Entre os principais resultados, destaca-se o melhor desempenho do operador de cruzamento *OX* em relação ao *PMX*, tanto em termos de qualidade das soluções quanto de estabilidade. Além disso, taxas de cruzamento mais elevadas mostraram-se benéficas, especialmente quando combinadas com o operador *OX*. A taxa de mutação apresentou um impacto mais sutil, contribuindo principalmente para a diversidade da população e para a redução da convergência prematura.

Outro ponto relevante foi a variabilidade dos resultados entre diferentes execuções, mesmo com os mesmos parâmetros, o que reforça a importância da análise estatística por meio de métricas como média, mediana e desvio padrão.

Por fim, os resultados obtidos demonstram que Algoritmos Genéticos são uma abordagem eficaz para o TSP, especialmente quando bem configurados, sendo capazes de produzir soluções de alta qualidade mesmo em problemas de maior complexidade.